

Análisis Técnico

link_bio

Arquitectura, Patrones y Conceptos de una
Aplicación Full-Stack
con Reflex, Python y Despliegue Cloud-Native

 Mayo 2026

 Alejandro Fuentes

 link-bio-blush-seven.vercel.app

 Python 3.11+ · Reflex 0.9.2 · React 19 · Starlette · Docker

Documento generado automáticamente con WeasyPrint

Índice de Contenidos

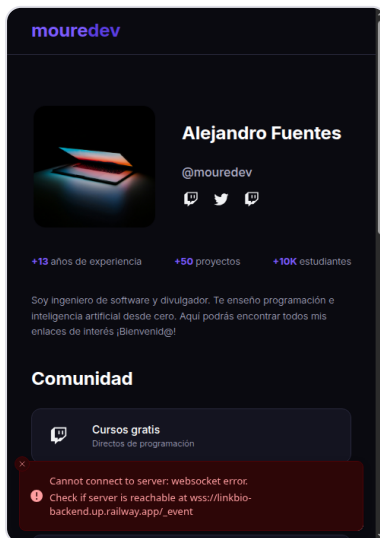
- | | | | |
|----|-----------------------------------|-----|-----------------------------------|
| 1. | Visión General | 7. | APIs REST y Servicios Externos |
| 2. | Stack Tecnológico | 8. | Sistema de Diseño (Design System) |
| 3. | Arquitectura — Despliegue Partido | 9. | Patrones de Código |
| 4. | Estructura del Proyecto | 10. | CI/CD y Despliegue |
| 5. | Flujo de Datos | 11. | Seguridad |
| 6. | Sistema de Estado Reactivo | 12. | Conclusiones |

1. Visión General

link_bio es una aplicación web full-stack construida con **Reflex** (un framework Python que compila a React) que funciona como un *link-in-bio* personalizado. En lugar de usar servicios comerciales como Linktree o Bio.link, esta implementación proporciona:

- Página principal con enlaces a redes sociales, cursos y recursos
- Detección de streaming en vivo desde Twitch en tiempo real
- Cálculo del próximo directo con conversión automática de husos horarios
- Sección de contenido destacado obtenido dinámicamente desde Supabase
- Despliegue partido en dos infraestructuras: frontend estático en Vercel + backend dinámico en Railway
- Integración continua mediante GitHub Actions con despliegue automático

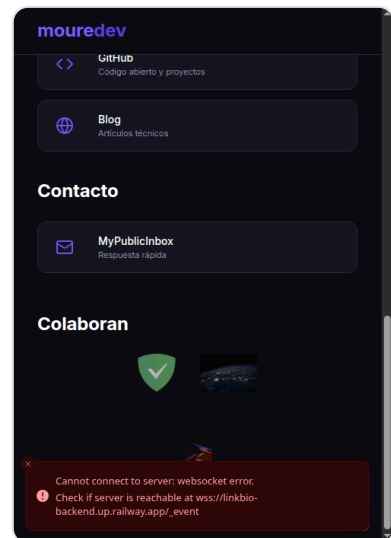
 **Propósito didáctico:** Este proyecto demuestra dominio de Python moderno (dataclasses, async/await, ZoneInfo, type hints), frameworks full-stack (Reflex sobre React + Starlette + Socket.IO), arquitectura de software (Atomic Design, DTOs, Singletons), consumo de APIs externas (OAuth2, REST), despliegue cloud-native (Docker, CI/CD, Vercel, Railway) y sistemas de diseño (design tokens, enums, CSS-in-JS).



Página principal (/) — Header con live indicator



Página de cursos (/cursos)



Footer y sección de patrocinadores

2. Stack Tecnológico — Fundamentos

2.1 Reflex — Full-Stack Python → React

Reflex es un framework que permite escribir aplicaciones web completas **exclusivamente en Python**. Su funcionamiento interno sigue este pipeline:

```
# Python → Compilación → React/JSX → Vite bundle → HTML+CSS+JS
# Python (rx.State) → WebSocket (Socket.IO) → Estado reactivo en frontend
# Python (app._api) → Starlette ASGI → Endpoints REST
```

Internamente, Reflex:

- 1. **Compila** los componentes Python (`rx.Component`) a JSX de React
- 2. Genera un **frontend SPA** (Single Page Application) con React Router 7
- 3. Provee un **backend Starlette/Uvicorn** para lógica de estado, APIs REST y WebSockets
- 4. Sincroniza estado entre frontend y backend vía **Socket.IO** (WebSocket con fallback a long-polling)

2.2 Dependencias Críticas

Librería	Versión	Rol Técnico
<code>reflex</code>	≥0.9.2.post1	Framework full-stack: <code>rx.App</code> , <code>rx.State</code> , <code>rx.page</code> , componentes reactivos
<code>python-dotenv</code>	≥1.2.2	Carga variables de entorno desde <code>.env</code> en tiempo de ejecución
<code>requests</code>	≥2.34.2	Cliente HTTP síncrono para la API de Twitch (Helix)
<code>supabase</code>	≥2.30.0	Cliente oficial Supabase: REST + PostgreSQL para contenido destacado
<code>configcat-client</code>	—	SDK de ConfigCat para feature flags y configuración remota

2.3 Frontend Bundles (gestionados por Reflex)

Reflex maneja automáticamente las dependencias JavaScript. El archivo `reflex.lock/package.json` revela:

Librería JS	Versión	Propósito
-------------	---------	-----------

React + ReactDOM	19.2.6	Framework UI con el nuevo reconciler (React 19)
react-router / react-router-dom	7.15.0	Enrutamiento SPA del lado del cliente
@radix-ui/themes	3.3.0	Sistema de componentes accesibles y estilizados
lucide-react	1.14.0	Librería de iconos SVG (código, globe, mail, etc.)
@emotion/react	11.14.0	CSS-in-JS, usado internamente por Reflex para estilos
tailwindcss	4.3.0	Framework de utilidades CSS (TailwindV4Plugin)
Vite	8.0.12	Bundler ultra-rápido para desarrollo y producción
socket.io-client	4.8.3	Comunicación bidireccional con el backend Reflex (WebSocket)
react-helmet	6.1.0	Gestión de etiquetas <head> para SEO (título, descripción)

3. Arquitectura — Despliegue Partido (Split Deployment)

El proyecto implementa una arquitectura **frontend-backend desacoplada**. El frontend estático se sirve desde Vercel (CDN global), mientras que el backend Python persiste en Railway como un contenedor Docker.

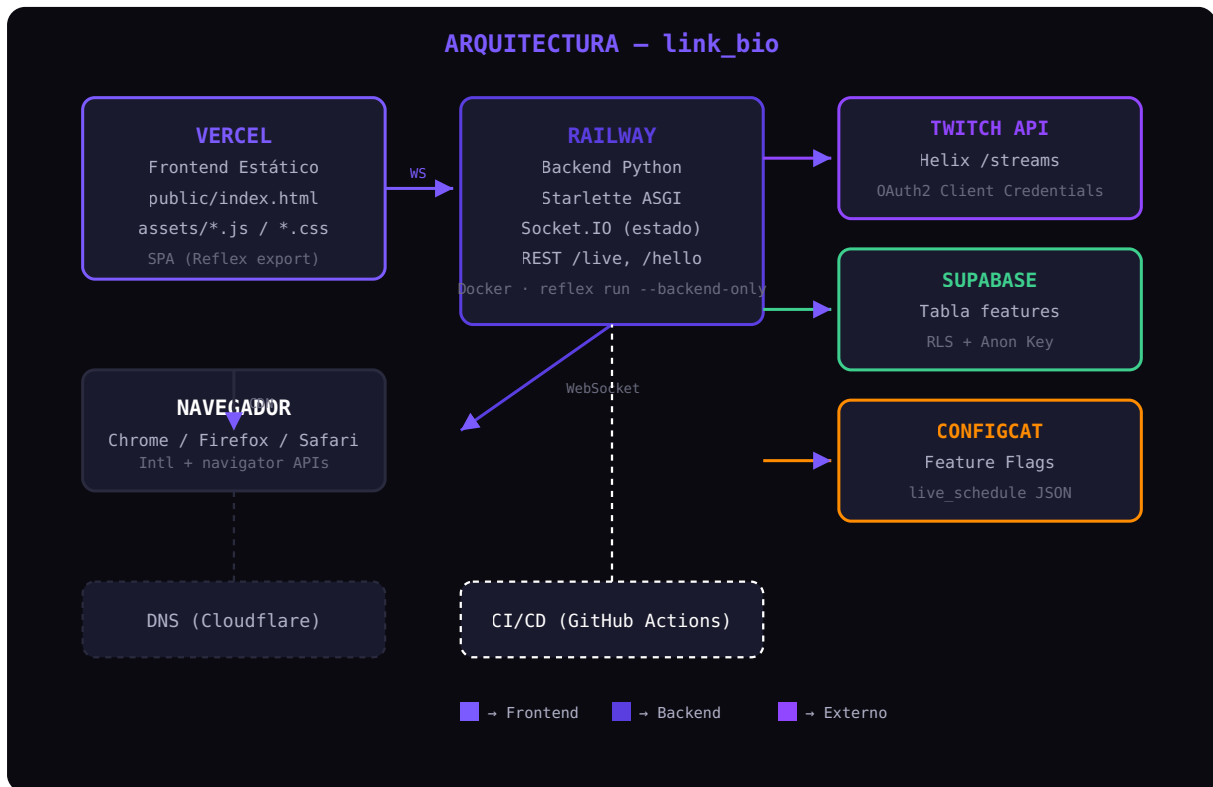


Diagrama 1: Arquitectura general — Frontend en Vercel, Backend en Railway, integración con Twitch, Supabase y ConfigCat

3.1 ¿Por qué separación frontend/backend?

- **Vercel** (Frontend): Sirve archivos estáticos (HTML, JS, CSS) desde 300+ ubicaciones CDN. Latencia mínima, sin servidor que mantener.
- **Railway** (Backend): Ejecuta un proceso Python persistente que mantiene estado en memoria, conexiones activas a APIs externas y WebSockets con los clientes.
- **Comunicación**: El frontend se comunica con el backend vía WebSocket (`socket.io`) para sincronización de estado Reflex, y vía REST para las rutas de API adicionales (`/hello` , `/live/{user}`).

4. Estructura del Proyecto

4.1 Árbol de Directorios

```

link_bio/
├── link_bio/                                # Paquete principal de la aplicación
│   └── link_bio.py                          # Entry point: rx.App, registro de páginas y
rutas API
│   └── routes.py                            # Enum Route: constantes tipadas para las
rutas
│   ├── utils.py                            # Utilidades: locale map, next_date(), SEO
│   ├── model/                              # Capa de dominio – dataclasses (DTOs)
│   │   ├── featured.py                    #   Featured: title, url, image
│   │   └── live.py                       #   Live: live, title, user
│   ├── styles/                             # Sistema de diseño (Design Tokens)
│   │   ├── colors.py                     #   Enum Color + TextColor (paleta completa)
│   │   ├── fonts.py                      #   Enum Font + FontWeight (tipografía)
│   │   └── styles.py                     #   BASE_STYLE, Size enum, style dicts
│   ├── components/                        # Componentes atómicos (Atomic Design → Atoms)
│   │   ├── navbar.py                     #   Barra de navegación sticky con logo + FAB
│   │   ├── footer.py                     #   Pie de página con copyright dinámico
│   │   └── link_button.py                 #   Botón de enlace reutilizable (icono +
texto)
│   ├── link_icon.py                       #   Icono social clickeable con hover scale
│   ├── featured_links.py                  #   Tarjeta de contenido destacado con imagen
│   ├── info_text.py                       #   Texto informativo (título bold + cuerpo)
│   ├── title.py                           #   Encabezado de sección estilizado
│   ├── ant_components.py                  #   Botón flotante (FAB) posición fija
│   └── views/                             # Vistas compuestas (Atomic Design →
Organisms)
│   ├── header.py                         #   Perfil: avatar, stats, live indicator, bio
│   └── index_links.py                     #   Enlaces del home + destacados
condicionales
│   ├── courses_links.py                  #   Enlaces de la página de cursos
│   └── sponsors.py                        #   Sección de patrocinadores con hover
opacity
│   ├── pages/                             # Páginas (Atomic Design → Templates)
│   │   ├── index.py                      #   Ruta / con on_load completo
│   │   └── courses.py                    #   Ruta /cursos
│   ├── state/                             # Estado global reactivo
│   │   └── pagesstate.py                 #   PagesState: 5 vars reactivas, 5 métodos
│   ├── api/                               # Adaptadores de servicios externos
│   │   ├── api.py                       #   Singletons + handlers Starlette
│   │   ├── TwitchAPI.py                  #   Cliente Twitch Helix (OAuth2 + streams)
│   │   ├── SupabaseAPI.py                #   Cliente Supabase (tabla features)
│   │   └── configcat.py                  #   Cliente ConfigCat (feature flags)
├── assets/                                # Estáticos: imágenes, CSS, iconos SVG
├── public/                                # Frontend compilado (reflex export)
├── rxconfig.py                            # Configuración de Reflex
├── pyproject.toml                         # Metadatos y dependencias Python
├── Dockerfile                             # Imagen Docker para Railway
├── vercel.json                            # Configuración Vercel (outputDirectory)
├── railway.json                           # Configuración Railway (Docker)
├── build.sh / remote_build.sh             # Scripts de build frontend
└── .github/workflows/deploy.yml           # CI/CD: GitHub Actions → Vercel

```


4.2 Atomic Design

El proyecto organiza sus componentes siguiendo la metodología **Atomic Design** de Brad Frost, que jerarquiza la UI en cinco niveles:

- **Átomos:** `link_button.py` , `link_icon.py` , `title.py` , `info_text.py` , `ant_components.py` — componentes indivisibles
- **Moléculas:** `navbar.py` (logo + float_button), `header.py` (avatar + stats + live indicator)
- **Organismos:** `index_links.py` (compone múltiples `link_button` + `title` + `featured_links`), `sponsors.py`
- **Plantillas:** Las páginas `index.py` y `courses.py` actúan como layouts
- **Páginas:** Instancias concretas con estado y datos reales

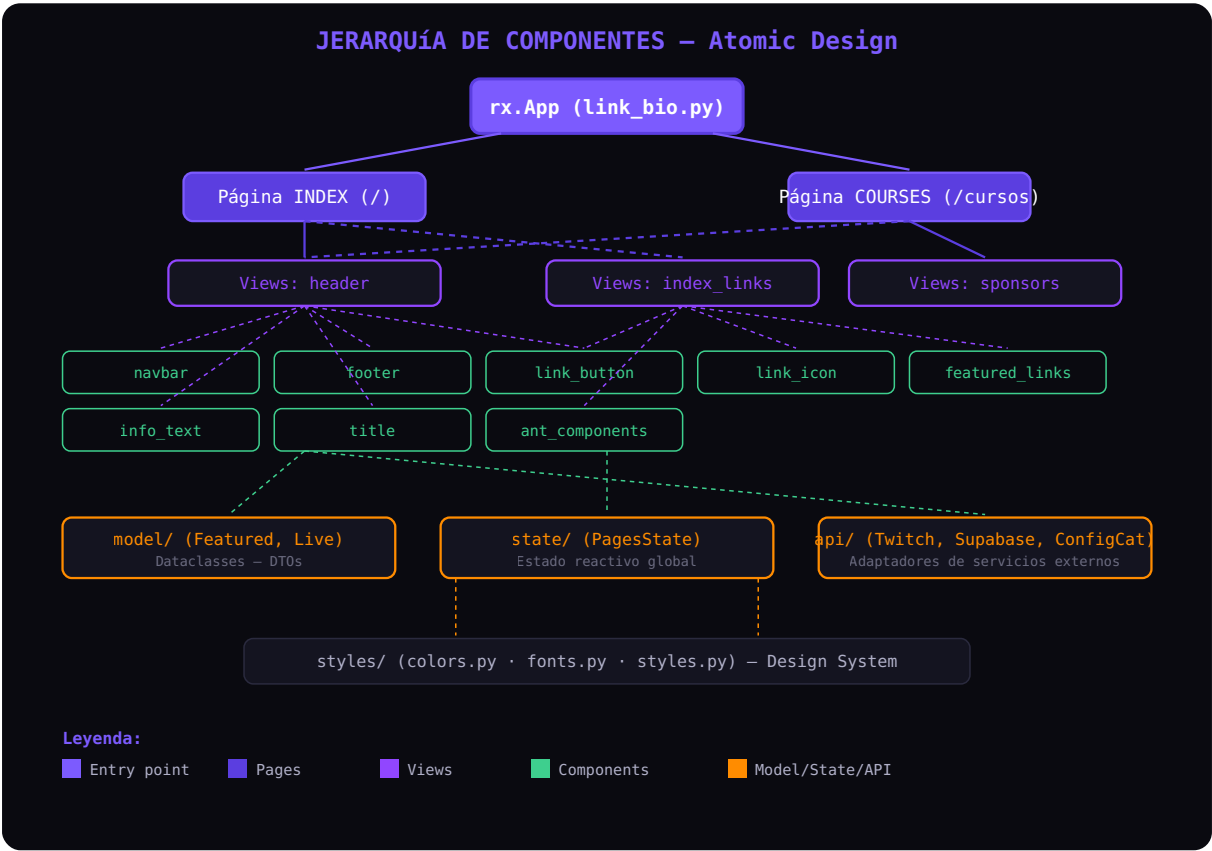


Diagrama 2: Jerarquía de componentes siguiendo Atomic Design — desde `rx.App` hasta los átomos y la capa `Model/State/API`

4.3 Separación de Responsabilidades (SoC)

Capa	Directorio	Responsabilidad
Dominio	<code>model/</code>	Dataclasses anémicas (puramente datos, sin lógica)
Infraestructura	<code>api/</code>	HTTP clients, servicios externos (Twitch, Supabase, ConfigCat)

Aplicación	<code>state/</code>	Estado global, orquestación de llamadas, lógica de negocio
Presentación	<code>views/</code>	Composición de componentes en secciones completas
UI	<code>components/</code>	Componentes reutilizables y atómicos
Routing	<code>pages/</code>	Páginas completas con decoradores @rx.page

5. Flujo de Datos

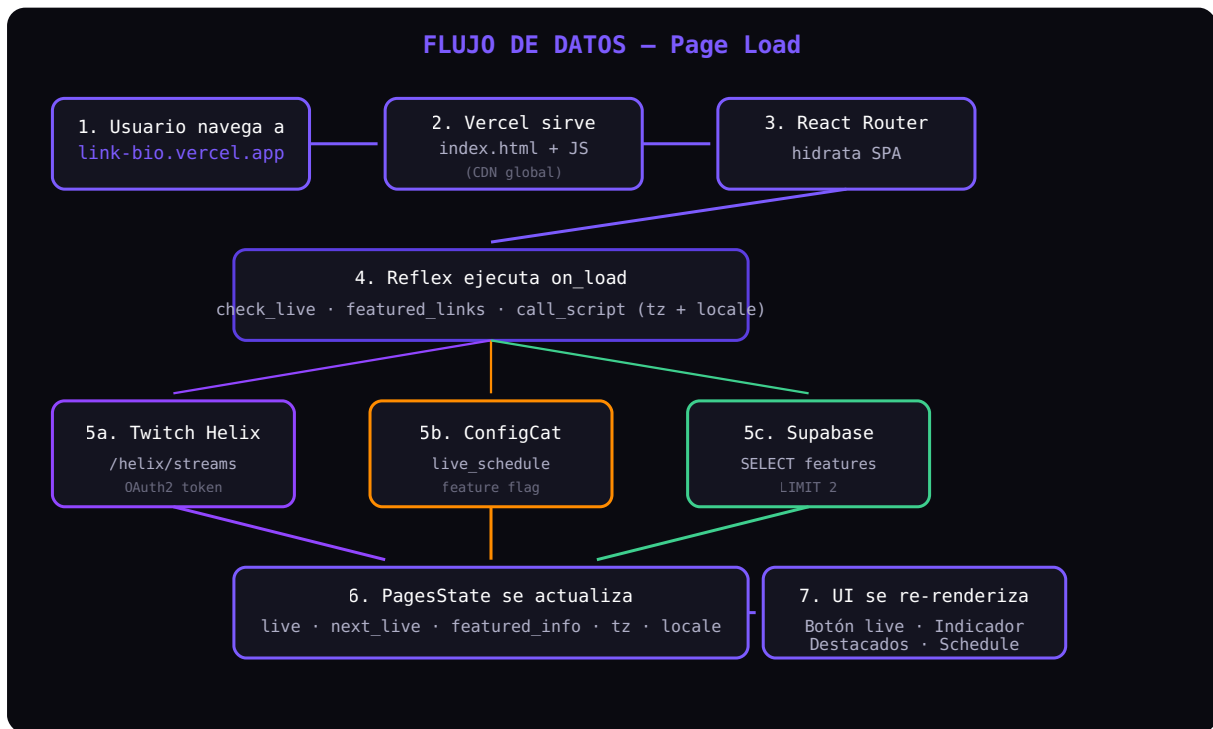


Diagrama 3: Flujo de datos completo desde la carga de página hasta la actualización de la UI

5.1 Carga de Página (Page Load)

Cuando un usuario visita la aplicación, el decorador `@rx.page` con el parámetro `on_load` ejecuta una secuencia de acciones asíncronas:

```

# link_bio/pages/index.py
@rx.page(
    title=utils.index_title,
    description=utils.index_description,
    on_load=[
        PagesState.check_live,          # Async: Twitch + ConfigCat
        PagesState.featured_links,      # Async: Supabase
        rx.call_script(                  # JS → captura timezone del browser
            "Intl.DateTimeFormat().resolvedOptions().timeZone",
            PagesState.set_timezone,
        ),
        rx.call_script(                  # JS → captura locale del browser
            "(navigator.language || navigator.languages[0])",
            PagesState.set_locale,
        ),
    ]
)

```

5.2 Captura de Zona Horaria y Locale

Se utiliza `rx.call_script` para ejecutar JavaScript nativo del browser y pasar el resultado a métodos del estado Reflex:

- `Intl.DateTimeFormat().resolvedOptions().timeZone` — API estandar del W3C que devuelve la zona horaria IANA del usuario (ej: "America/Argentina/Buenos_Aires")
- `navigator.language || navigator.languages[0]` — Código de idioma/región del navegador (ej: "es-AR", "en-US")

Concepto: *Client-side detection* — el servidor no puede conocer la zona horaria del usuario, por lo que se delega al cliente la captura de estos valores.

5.3 Verificación de Live en Twitch

```
# Flujo OAuth2 Client Credentials
def generate_token(self):
    response = requests.post(
        "https://id.twitch.tv/oauth2/token",
        data={
            "client_id": self.CLIENT_ID,
            "client_secret": self.CLIENT_SECRET,
            "grant_type": "client_credentials"
        }
    )
    data = response.json()
    self.token = data["access_token"]
    self.token_exp = time.time() + data["expires_in"]

def live(self, user) → Live:
    response = requests.get(
        f"https://api.twitch.tv/helix/streams?user_login={user}",
        headers={
            "Client-Id": self.CLIENT_ID,
            "Authorization": f"Bearer {self.token}",
        }
    )
    if response.json().get("data"):
        data = response.json()["data"]
        return Live(live=True, title=data[0]["title"], user=user)
```

Conceptos aplicados:

- **OAuth 2.0 Client Credentials Grant:** Flujo máquina-a-máquina (sin intervención de usuario). Twitch emite un `access_token` con tiempo de expiración (`expires_in`).
- **Token caching:** `self.token_exp` almacena `time.time() + expires_in` para evitar regenerar el token en cada request.
- **Estrategia defensiva:** `if not self.token_valid(): self.generate_token()` — regenera solo cuando expira.

- **Graceful degradation:** Los `try/except` en `check_live` aseguran que si Twitch falla, la UI muestra estado offline sin romperse.

5.4 Cálculo del Próximo Directo (Time Zone Conversion)

```
def next_date(dates: dict, user_tz: str, locale: str) → str:
    madrid_tz = ZoneInfo("Europe/Madrid")
    now_madrid = datetime.now(madrid_tz)
    current_weekday = now_madrid.weekday()

    for i in range(7): # Próximos 7 días
        day = str((current_weekday + i) % 7)
        if day not in dates or not dates[day]:
            continue
        hour, minute = map(int, dates[day].split(":"))
        event_madrid = now_madrid.replace(
            hour=hour, minute=minute
        ) + timedelta(days=i)

        if user_tz and user_tz != "Europe/Madrid":
            event_user = event_madrid.astimezone(ZoneInfo(user_tz))
            country = _country_from_locale(locale)
            return f"{formatted_date} a las {formatted_time} (hora de {country})"
```

Conceptos aplicados:

- **ZoneInfo (PEP 615):** Base de datos IANA de husos horarios, integrada nativamente desde Python 3.9+. Soporta DST (horario de verano) automáticamente.
- **Conversión de husos:** `astimezone()` convierte entre zonas horarias respetando reglas de cambio estacional.
- **Algoritmo de búsqueda hacia adelante:** Itera hasta 7 días desde hoy buscando el próximo evento programado. Salta el día actual si la hora ya pasó.
- **Locale mapping:** Diccionario `LOCALE_MAP` con 50+ entradas para mostrar el país en español.

5.5 Featured Links desde Supabase

```
# link_bio/api/SupabaseAPI.py
class SupabaseAPI:
    def featured(self) → list[Featured]:
        response = self.supabase.table("features")\
            .select("*").limit(2).execute()
        return [
            Featured(
                title=item["title"],
                url=item["url"],
                image=item["image"]
            )
            for item in response.data
        ]
```

Conceptos:

- **Row Level Security (RLS):** Supabase usa la publishable key (anon key) que solo permite consultas seguras definidas por políticas RLS en PostgreSQL.
- **DTO Pattern:** Los datos crudos de la API REST se transforman a dataclasses `Featured` (Data Transfer Object) para desacoplar la capa de datos de la capa de presentación.
- **Query limitada:** `.limit(2)` evita sobrecarga de red — es un link-in-bio, no un catálogo.

6. Sistema de Estado Reactivo (PagesState)

```
# link_bio/state/pagesstate.py
class PagesState(rx.State):
    live: Live = Live(live=False, title=None, user="")
    next_live: str = ""
    featured_info: list[Featured] = []
    client_tz: str = "Europe/Madrid"
    client_locale: str = "es-ES"

    async def check_live(self):
        try:
            self.live = TWITCH_API.live("ohnePixel")
        except Exception:
            self.live = Live(live=False, title=None, user="ohnePixel")
        try:
            schedule_dict = await schedule()
            self.next_live = utils.next_date(
                schedule_dict, self.client_tz, self.client_locale
            )
        except Exception:
            self.next_live = ""

    async def featured_links(self):
        try:
            self.featured_info = SUPABASE_API.featured()
        except Exception:
            self.featured_info = []

    def _recalc_schedule(self):
        # Recalcula next_live sin llamar a Twitch de nuevo
        schedule_dict = CONFIGCAT_API.schedule()
        self.next_live = utils.next_date(
            schedule_dict, self.client_tz, self.client_locale
        )
```

6.1 Conceptos Clave de Reflex State

Reactividad

Cada variable de estado es reactiva. Cuando cambia, todos los componentes que la referencian se re-renderizan automáticamente sin intervención manual.

Sincronización C-S

Reflex mantiene el estado en el servidor Python y replica los cambios al frontend vía WebSocket (Socket.IO).

Métodos Asíncronos

`check_live` y `featured_links` son `async` porque hacen HTTP requests. Reflex maneja la concurrencia automáticamente.

Event Lifecycle

`on_load` →
`call_script` (JS) →
`set_timezone()` →
`_recalc_schedule()` →
UI update.

Graceful Degradation

Cada llamada externa está envuelta en `try/except`. Si Twitch, Supabase o ConfigCat fallan, la app sigue funcionando con valores por defecto.

Re-cálculo parcial

`_recalc_schedule()` actualiza `next_live` sin repetir la llamada a Twitch cuando solo cambian `tz` o `locale`.

7. APIs REST y Servicios Externos

7.1 Registro de Rutas

El archivo `link_bio.py` registra rutas REST directamente en el backend Starlette:

```
# link_bio/link_bio.py
app._api.add_route("/hello", hello)          # GET → "Hola"
app._api.add_route("/live/{user}", live)     # GET → JSON {live, title}
app._api.add_route("/user/{user}", live)     # GET → mismo handler
```

7.2 Handlers Starlette

```
from starlette.requests import Request
from starlette.responses import JSONResponse

async def live(request: Request) → JSONResponse:
    user = request.path_params.get("user", "")
    live_status = TWITCH_API.live(user)
    return JSONResponse(
        content={"live": live_status.live, "title": live_status.title}
    )
```

Conceptos:

- **Path parameters:** Starlette extrae `{user}` de la URL y lo expone en `request.path_params`
- **ASGI:** Starlette es un framework ASGI, el sucesor moderno de WSGI. Soporta async nativo.
- **Content Negotiation:** `JSONResponse` automáticamente setea `Content-Type: application/json`

7.3 Servicios Externos

Servicio	Protocolo	Función
Twitch Helix	OAuth2 + REST	Verificar si "ohnePixel" está en vivo
Supabase	REST + PostgreSQL	Obtener contenido destacado (tabla <code>features</code>)
ConfigCat	Feature Flag SDK	Obtener horario de directos (<code>live_schedule</code> JSON)

8. Sistema de Diseño (Design System)

8.1 Paleta de Colores

```
class Color(Enum):
    PRIMARY = "#7C5BFE"      # Púrpura principal
    SECONDARY = "#5B3EE0"    # Púrpura oscuro (hover)
    BACKGROUND = "#0A0A10"   # Casi negro (fondo)
    CONTENT = "#141420"      # Gris oscuro (cards)
    BORDER = "#2A2A3E"       # Borde sutil
    PURPLE = "#9146ff"       # Púrpura Twitch (live)
```

8.2 Tipografía y Tamaños

```
class Font(Enum):
    DEFAULT = TITLE = LOGO = "Inter" # Single font family

class FontWeight(Enum):
    LIGHT = "300"; MEDIUM = "500"; BOLD = "700"

class Size(Enum):
    ZERO = "0em"; SMALL = "0.5em"; MEDIUM = "0.8em"
    DEFAULT = "1em"; LARGE = "1.5em"; BIG = "2em"; VERYBIG = "3em"
```

Conceptos aplicados:

- **Enum pattern:** Centraliza colores, fuentes y tamaños en un solo lugar. Elimina magic strings.
- **Dark theme by default:** Fondo `#0A0A10` con texto blanco/gris — diseño oscuro nativo sin toggle.
- **Design tokens:** Cada tamaño tiene un nombre semántico, no un valor absoluto.
- **Unidades em:** Relativas al tamaño de fuente del contenedor. Garantizan accesibilidad (escalán con zoom).

8.3 BASE_STYLE — Estilos Globales

```
BASE_STYLE = {
  "font_family": Font.DEFAULT.value,
  "background_color": Color.BACKGROUND.value,
  rx.button: {
    "transition": "all 0.3s ease",
    "_hover": {
      "transform": "translateY(-2px)",
      "box_shadow": "0 8px 25px rgba(124, 91, 254, 0.15)",
    },
  },
}
```

Conceptos:

- **CSS-in-JS via dicts Python:** Reflex convierte estos diccionarios a objetos `style` de React o clases CSS.
- **Pseudoclases:** `_hover` se transpila a `:hover` en CSS.
- **Jerarquía de estilos:** Los estilos de `rx.button` se aplican a **todos** los botones globalmente.

8.4 Animación CSS Personalizada

```
/* assets/css/styles.css */
@keyframes blinker {
  0% { opacity: 1; }
  50% { opacity: 0; }
  100% { opacity: 1; }
}
.blink {
  animation: blinker 1s linear infinite;
}
```

Esta animación se aplica al indicador de live (punto púrpura sobre el avatar) mediante `class_name="blink"` en el `rx.box`. El efecto de parpadeo infinito señala visualmente que el streamer está en vivo.

9. Patrones de Código Destacables

9.1 Singleton Pattern

```
# Los tres clientes de API se instancian una sola vez a nivel de módulo
TWITCH_API = TwitchAPI()      # Instancia única global
CONFIGCAT_API = ConfigCatAPI()
SUPABASE_API = SupabaseAPI()
```

9.2 Strategy Pattern

```
# Decide dinámicamente el tipo de icono según la extensión
icon_component = rx.image(src=icon, ...) if icon.endswith(
    ".svg", ".png", ".jpg")
) else rx.icon(tag=icon, ...)
```

9.3 Conditional Rendering

```
# rx.cond() ≡ operador ternario de React (condition ? A : B)
rx.cond(
    live.live,
    link_button("En directo en Twitch", live.title, ...),
    link_button("Próximo directo", next_live, ...),
)
```

9.4 ForEach Loop

```
# rx.foreach() itera sobre una lista reactiva
rx.foreach(featured, lambda item: featured_links(item))
```

9.5 Callback Pattern (JS → Python)

```
# Ejecuta JS en el browser y pasa el resultado al estado Python
rx.call_script(
    "Intl.DateTimeFormat().resolvedOptions().timeZone",
    PageState.set_timezone, # Callback: resultado JS como argumento
)
```

9.6 DTO Pattern

```
@dataclass
class Live:
    live: bool
    title: str | None
    user: str

@dataclass
class Featured:
    title: str
    url: str
    image: str
```

Conceptos:

- **Python 3.10+ Union Type:** `str | None` en lugar de `Optional[str]` (PEP 604)
- **Tipado estático:** Permite que el IDE y type checkers (mypy, pyright) verifiquen el código en tiempo de desarrollo
- **Desacoplamiento:** Los datos crudos de APIs externas se transforman a DTOs antes de llegar a la capa de presentación

10. CI/CD y Despliegue

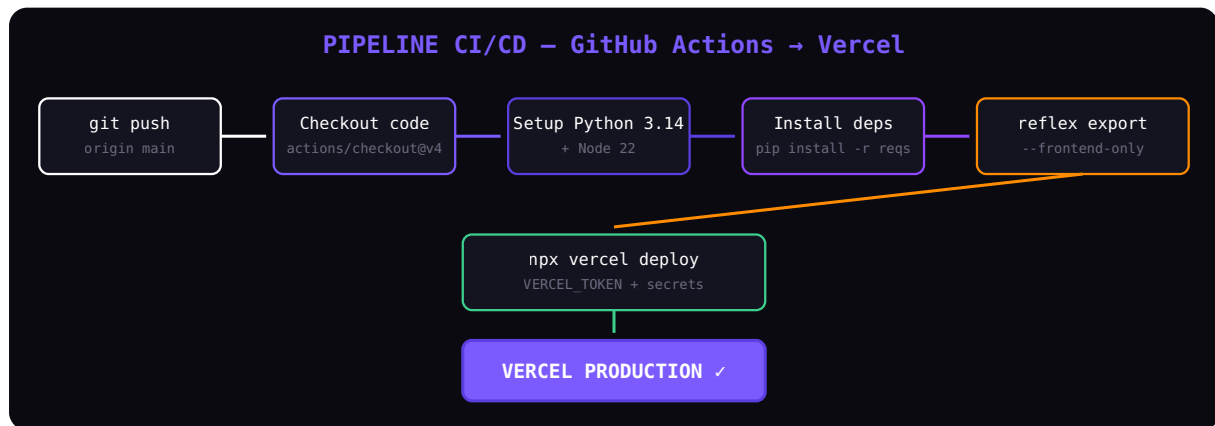


Diagrama 4: Pipeline CI/CD — desde git push hasta deploy en Vercel

10.1 GitHub Actions Workflow

```

# .github/workflows/deploy.yml
name: Build & Deploy to Vercel
on:
  push:
    branches: [main]
  workflow_dispatch:
jobs:
  build-and-deploy:
    steps:
      - actions/checkout@v4
      - setup-python@v5 (3.14)
      - setup-node@v4 (22)
      - pip install -r requirements.txt
      - reflex init
      - reflex export --frontend-only
      - unzip frontend.zip -d public
      - npx vercel deploy --prod
  
```

10.2 Dockerfile (Backend Railway)

```

FROM python:3.13-slim
WORKDIR /app
RUN apt-get update -y && apt-get install -y gcc unzip curl
RUN pip install --no-cache-dir uv reflex
COPY . .
RUN uv pip install --system -r requirements.txt
RUN reflex init || true
CMD reflex run --env prod --backend-only
  
```

Conceptos aplicados:

- **CI/CD:** Cada push a main gatilla build automático + deploy sin intervención manual.
- **Infrastructure as Code:** `vercel.json` , `railway.json` , `Dockerfile` definen la infraestructura declarativamente.
- **uv:** Package manager escrito en Rust por Astral. Resuelve dependencias ~10-100x más rápido que pip.
- **STOPSIGNAL SIGKILL:** Forza terminación inmediata del contenedor (necesario en Railway para reinicios rápidos).
- **workflow_dispatch:** Permite ejecutar el pipeline manualmente desde la UI de GitHub Actions.

11. Seguridad — Análisis

11.1 Riesgos Identificados

Riesgo	Archivo	Impacto	Recomendación
.env con secrets en disco	<code>.env</code>	 ALTO	Usar secrets manager (GitHub Secrets, Railway Secrets, Vercel Environment Variables)
Supabase anon key	<code>.env</code>	 BAJO	Es publishable por diseño. La seguridad real está en las políticas RLS de Supabase.
Sin rate limiting	<code>api/api.py</code>	 MEDIO	Implementar throttling con slowapi o middleware Starlette.
GitHub Secrets en CI/CD	<code>deploy.yml</code>	 BAJO	VERCEL_TOKEN, VERCEL_ORG_ID, VERCEL_PROJECT_ID se inyectan como secrets → buenas práctica ✓

11.2 Buenas Prácticas Aplicadas

- **Resiliencia:** `try/except` en todas las llamadas a servicios externos (Twitch, Supabase, ConfigCat). Si fallan, la app usa valores por defecto.
- **Token caching:** TwitchAPI almacena el token con tiempo de expiración, evitando regenerarlo en cada request.
- **Null safety:** Validación de `None` antes de crear clientes Supabase/ConfigCat si faltan credenciales.
- **.env en .gitignore:** Aunque el archivo existe en disco, está listado en `.gitignore` para evitar commits accidentales.

12. Conclusiones

El proyecto **link_bio** demuestra dominio de un stack tecnológico moderno y buenas prácticas de ingeniería de software:

Python Moderno dataclasses (PEP 557), ZoneInfo (PEP 615), async/await, type hints con union syntax (PEP 604).	Frameworks Full-Stack Reflex como capa de abstracción sobre React 19 + React Router 7 + Starlette + Socket.IO + Vite 8 + Tailwind 4.	Arquitectura de Software Atomic Design, DTOs, Singletons, Separación de Responsabilidades (SoC), composición vs herencia.
APIs Externas OAuth2 Client Credentials, REST clients con caching, feature flags remotos, Row Level Security en Supabase.	Despliegue Cloud-Native Docker, CI/CD con GitHub Actions, frontend estático en Vercel, backend containerizado en Railway.	Sistemas de Diseño Design tokens con enums, CSS-in-JS, animaciones keyframes, tipografía variable, tema oscuro nativo.
Web Fundamentals Time zone handling (IANA), locale detection (Intl API), SEO (meta tags, sitemap XML), client-side detection.	Resiliencia Graceful degradation en todas las integraciones, defaults seguros, caching estratégico, manejo defensivo de errores.	Tipado Estático Type hints en todas las funciones y métodos. Las dataclasses usan tipos explícitos permitiendo verificación estática.

 **Nota final:** Áreas de mejora: (1) mover secrets a un vault externo, (2) añadir tests automatizados, (3) implementar rate limiting en las APIs, (4) corregir los stubs de métodos `check_schedule` y `check_featured` que no existen en `PagesState`, (5) unificar el icono de GitHub (actualmente usa el SVG de Twitch).